

贝尔赛克指纹模组 通讯协议总览

BIOSEC CONFIDENTIAL

目 录

1	八字节格式	1
2	大于八字节，数据分为数据包头部+数据包	2
3	命令定义	2
3.1	用户注册简介	2
3.1.1	多次按压注册（3C3R）（CMD/ACK 都是 8 字节）	3
3.1.2	多次按压注册（NCNR）（CMD/ACK 都是 8 字节）	4
3.2	删除特定用户和特定权限用户（CMD/ACK 都是 8 字节）	4
3.3	删除所有用户（CMD/ACK 都是 8 字节）	5
3.4	获取用户总数（CMD/ACK 都是 8 字节）	5
3.5	获取用户权限（CMD/ACK 都是 8 字节）	6
3.6	1 比 1 比对（CMD/ACK 都是 8 字节）	6
3.7	1 比 N 比对和特定权限用户比对（CMD/ACK 都是 8 字节）	7
3.8	获取未使用的用户 ID（CMD/ACK 都是 8 字节）	8
3.9	设置串口波特率（CMD/ACK 都是 8 字节）	8
3.10	读取图像并提取特征值（CMD 是 8 字节/ ACK >8 字节）	9
3.11	获取图像（CMD 是 8 字节/ ACK >8 字节）	9
3.12	获取模块版本信息（CMD 是 8 字节/ ACK >8 字节）	10
3.13	设置/获取比对等级（CMD/ACK 都是 8 字节）	10
3.14	设置/获取注册模式（CMD/ACK 都是 8 字节）	11
3.15	手指检测功能（CMD/ACK 都是 8 字节）	12

3.16 获取指纹模组唯一 ID 号 (CMD/ACK 都是 8 字节) 12

3.17 配置注册功能 (CMD/ACK 都是 8 字节) 12

3.18 光圈控制命令 (CMD/ACK 都是 8 字节) 14

BIOSEC CONFIDENTIAL

通信协议总览

主机端和模组从机端通过 F5 开头，F5 结尾的 CMD 包和 ACK 包来通讯。主机端发送 CMD 命令包（8 字节）和 CMD 数据包（一般大于 8 字节），而从机端反馈 ACK 命令包（8 字节）和 ACK 数据包（一般大于 8 字节）。

1 八字节格式

数据格式如下：

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	TYPE	P1	P2	P3	0	CHK	0xF5
ACK	0xF5	TYPE	Q1	Q2	Q3	0	CHK	0xF5

Notes:

TYPE : CMD/ACK 类型

P1, P2, P3 : CMD 参数

Q1, Q2, Q3 : ACK 参数

Q3 总是用来返回命令执行的结果：

#define ACK_SUCCESS 0x00 //执行成功

#define ACK_FAIL 0x01 //执行失败

#define ACK_FULL 0x04 //数据库满

#define ACK_NOUSER 0x05 //没有这个用户

#define ACK_USER_EXIST 0x07 //用户已存在

#define ACK_TIMEOUT 0x08 //图像采集超时

#define ACK_HARDWAREERROR 0x0A //硬件错误

#define ACK_IMAGEERROR 0x10 //图像错误

#define ACK_BREAK 0x18 //终止当前指令

#define ACK_IMAGEFAIL 0x84 //指纹图像不合格

#define ACK_ALGORITHMFAIL 0x11 //贴膜攻击检测

#define ACK_HOMOLOGYFAIL 0x12 //同源性校验错误

CHK : 校验和，是第 2 个字节到第 6 个字节的异或校验和。

2 大于八字节，数据分为数据包头部+数据包

数据包头部格式：

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	TYPE	Hi (Len)	Low (Len)	0	0	CHK	0xF5
ACK	0xF5	TYPE	Hi (Len)	Low (Len)	Q3	0	CHK	0xF5

注意：

Q3 的定义可以在上面找到

Len：数据包中 Real Data 的数据长度，两字节组成

Hi (Len)：真实数据包长度的高字节

Low (Len)：真实数据包长度的低字节

CHK：校验和，是第 2 个字节到第 6 个字节的异或校验和

数据包格式：

Byte	1	2...Len+1	Len+2	Len+3
CMD	0xF5	Real Data	CHK	0xF5
ACK	0xF5	Real Data	CHK	0xF5

注意：

Len：Real Data 的数据长度

CHK：校验和，是第 2 个字节到第 Len +1 个字节的异或校验和数据包紧跟着数据包头部发送出来。

3 命令定义

3.1 用户注册简介

用户注册共有 2 种主流的注册方式，即 3C3R、NCNR(默认的标准版本为 3C3R 的模式)。

以下分别介绍两种模式：

1. 3C3R 的注册方式，在多次命令多次返回的注册方式中是比较普遍的注册方式，因为其同时兼顾了用户的操作复杂度和用户的使用随意度。
2. NCNR 的注册方式，用户可以根据应用调整注册次数，类似 3C3R 注册方式，一般以 N=3 时为最佳用户注册体验度。

注 1：模组默认带有贴膜攻击检测功能，注册时手指不能一直按压注册，建议使用 3C3R 的注册方式（多次注册可使用 NCNR 注册方式），并引导用户抬起手指注册。

注 2：在所有类型的注册过程中，若收到 ACK 命令包的 TYPE 字段（参见前面命令格式描述）是 0x03 则表示此次注册结束，若该 ACK 命令包的 Q3 是 0，就表示此次注册成功且该

ACK 的 Q1 和 Q2 表示用户 ID 的高 8 位和低 8 位。具体可以参看以下详解。

3.1.1 多次按压注册（3C3R）（CMD/ACK 都是 8 字节）

第一次注册发送的命令和返回

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x01	UserID(H igher8bi ts)	UserID(L ower8 bits)	User Role (1/2/3)	0	CHK	0xF5
ACK	0xF5	0x01	UserID(H igher8bi ts) 0	UserID(L ower8 bits) 0	ACK_SUCCESS ACK_FAIL ACK_FULL ACK_TIMEOUT ACK_BREAK ACK_IMAGEFAIL ACK_HARDWAREERROR ACK_IMAGEERROR	0	CHK	0xF5

第一次命令，需要设置 ID 值和权限，User ID 为 1-10000, 若设置 ID 为 0，则模组自动分配一个未使用的 ID 给用户，但用户总数量不会超过单个模组所能存储的最大数量。

第二次注册发送的命令和返回

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x02	0	0	0	0	CHK	0xF5
ACK	0xF5	0x02 0x03	0	0	ACK_SUCCESS ACK_FAIL ACK_FULL ACK_TIMEOUT ACK_BREAK ACK_IMAGEFAIL	0	CHK	0xF5

若此时返回 ACK 的第二个字节是 0x03 就表示注册被提前终止，注册终止原因可通过第五个字节获得，需要用户重新开始注册。

第三次注册发送的命令和返回

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x03	0	0	0	0	CHK	0xF5
ACK	0xF5	0x03	UserID(H igher8bi ts) 0	UserID(L ower8 bits) 0	ACK_SUCCESS ACK_FAIL ACK_FULL ACK_TIMEOUT ACK_BREAK ACK_IMAGEFAIL ACK_ALGORITHMFAIL ACK_HOMOLOGYFAIL	0	CHK	0xF5

在三次注册过程的任意时刻返回注册失败，则需要重新开始注册，例如：用户第三次命

令发出后，没有返回 ACK_SUCCESS，则用户需要从第一次注册开始重新发送注册指令进行注册。只有在注册成功时，第三和第四字符才表示是用户 ID 的高 8 位和低 8 位。

3.1.2 多次按压注册 (NCNR) (CMD/ACK 都是 8 字节)

NCNR 设置后发送注册指令，发送次数与 N 有关。

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x01	UserID(H igher8bi ts)	UserID(L ower8 bits)	ROLE	0	CHK	0xF5
...	0xF5	0x01	0	0	0	0	CHK	0xF5
...	0xF5	0x01	0	0	0	0	CHK	0xF5
ACK	0xF5	0x03	UserID(H igher8bi ts)	UserID(L ower8 bits)	ACK_SUCCESS ACK_FAIL ACK_FULL ACK_TIMEOUT ACK_BREAK ACK_IMAGEFAIL ACK_ALGORITHMFAIL ACK_HOMOLOGYFAIL	0	CHK	0xF5

当 N>3 时，注册连续发送的指令为 F5 01 00 00 00 00 01 F5，返回值除最后一次以外都以 F5 01 00 00 00 00 01 F5 返回。最后一次返回注册成功/失败结果或检测 TYPE 位为 03 时判断注册成功/失败结果。

3.2 删除特定用户和特定权限用户 (CMD/ACK 都是 8 字节)

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x04	User ID(Higher 8 Bits)	User ID(Low 8 Bits)	Role	0	CHK	0xF5
ACK	0xF5	0x04	0	0	ACK_SUCCESS ACK_NOUSER	0	CHK	0xF5

说明：

删除指纹库内指定 ID 的指纹，以及按权限删除用户 ID 功能。

示例 1：

(删除 0001 号用户) 通过串口发送删除指定 ID 指令

F5 04 00 01 00 00 05 F5

模组收到指令后，模组搜寻指纹库内有没有 0001 号用户，搜寻成功之后就把 0001 号指纹删除，串口返回

F5 04 00 00 00 00 04 F5 (删除成功提示)

搜寻不到返回失败提示

F5 04 00 00 05 00 01 F5 (用户不存在)

示例 2:

按权限 02 删除所有权限为 02 的用户

发送: F5 04 00 00 02 00 06 F5

返回: F5 04 00 00 00 00 04 F5(删除成功提示)

示例 3:

删除指定用户 (00 0A 号用户) 与权限为 03 的所有用户

发送: F5 04 00 0A 03 00 0D F5

返回: F5 04 00 00 00 00 04 F5(删除成功提示)

3.3 删除所有用户 (CMD/ACK 都是 8 字节)

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x05	0	0	0	0	CHK	0xF5
ACK	0xF5	0x05	0	0	ACK_SUCCESS	0	CHK	0xF5

说明:

删除指纹库内所有指纹。

示例:

通过串口发送删除所有用户指令

F5 05 00 00 00 00 05 F5

删除成功提示 F5 05 00 00 00 00 05 F5

注: 进行删除时, 必须要等待模组有返回值后再对模组进行其它操作。

模组收到指令后, 把模组内所有指纹删除, 此命令不会返回失败, 除该指令用作打断其他指令。

例如:

在注册过程中发送

F5 05 00 00 00 00 05 F5

会造成注册终止, 同时该条清除指令也不会执行

3.4 获取用户总数 (CMD/ACK 都是 8 字节)

Bytes	1	2	3	4	5	6	7	8
CMD	0xF5	0x09	0	0	0	0	CHK	0xF5
ACK	0xF5	0x09	Usercount (higher8 bits)	Usercount (lower8 bits)	ACK_SUCCESS	0	CHK	0xF5

说明:

获取指纹库内已有用户数量

示例: 通过串口发送获取用户总数命令

F5 09 00 00 00 00 09 F5

模组收到指令后, 搜寻模组存在的用户数量。 通过串口把用户数量发送出来。

F5 09 00 01 00 00 08 F5

表示指纹库内有 0001 个用户。

3.5 获取用户权限(CMD/ACK 都是 8 字节)

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x0A	UserID (Higher 8bits)	UserID (Lower 8bits)	0	0	CHK	0xF5
ACK	0xF5	0x0A	0	0	User role(1/2/3) ACK_NOUSER	0	CHK	0xF5

说明:

用来检测 ID 在指纹库内是否已经使用。

示例: (获取 0001 号用户的权限) 通过串口发送

F5 0A 00 01 00 00 0b F5

模组收到指令后, 若指纹库内不存在 0001 号用户
则返回

F5 0A 00 00 05 00 0F F5

若存在则返回

F5 0A 00 00 01 00 0B F5

3.6 1 比 1 比对(CMD/ACK 都是 8 字节)

Bytes	1	2	3	4	5	6	7	8
CMD	0xF5	0x0B	UserID (higher 8 bits)	UserID (lower 8 bits)	0	0	CHK	0xF5
ACK	0xF5	0x0B	0	0	ACK_SUCCESS ACK_FAIL ACK_TIMEOUT ACK_BREAK ACK_HARDWAREERROR ACK_IMAGEERROR	0	CHK	0xF5

说明:

采集指纹与指纹库指定 ID 用户比较。

示例:

(和 0001 号用户的比对) 通过串口发送指令

F5 0B 00 01 00 00 0A F5

模组收到指令后, 搜寻指纹库内 0001 号用户,
若不存在, 直接返回

F5 0B 00 00 05 00 0E F5

若存在，模组指纹灯点亮，等待用户采集指纹。 用户指纹没有按压 8S 后串口返回

F5 0B 00 00 08 00 03 F5

用户指纹按压之后，比对失败串口返回

F5 0B 00 00 01 00 0A F5

比对成功返回

F5 0B 00 00 00 00 0B F5

若在采集中收到其他指令，则终止当前的命令，且此条造成对比命令终止的命令也不会执行，返回

F5 0B 00 00 18 00 13 F5

3.7 1 比 N 比对和特定权限用户比对 (CMD/ACK 都是 8 字节)

Bytes	1	2	3	4	5	6	7	8
CMD	0xF5	0x0C	0	0	Role	0	CHK	0xF5
ACK	0xF5	0x0C	UserID (higher 8 bits)	UserID (lower 8 bits)	User role (1/2/3)	0	CHK	0xF5

说明：

采集指纹与指纹库所有用户比对，以及按权限比对功能。 比对成功返回第一个比对成功的用户 ID 和权限 Role。如果比对不成功，那么返回的 ID 值为 00 00。

示例 1:

通过串口发送指令

F5 0C 00 00 00 00 0C F5

模组收到指令后，等待用户输入指纹。

若没有输入指纹，模组超时退出，串口返回超时错误

F5 0C 00 00 08 00 04 F5

若输入指纹，模组搜寻库内指纹与刚采集的指纹进行比对，若比对失败，串口返回

F5 0C 00 00 00 00 0C F5

若比对成功，返回第一个比对成功用户的 ID:

F5 0C 2bytesID 1byteRole 00 CHK F5

(如: F5 0C 00 05 01 00 08 F5)

示例 2:

按权限 02 比对指纹 (用户 ID: 00 0A, 权限值: 02)

发送: F5 0C 00 00 02 00 0E F5 该指令只会比对权限值为 02 的所有指纹

返回: F5 0C 00 0A 02 00 04 F5 比对成功返回对应用户 ID 和权限值

返回: F5 0C 00 00 02 00 0E F5 比对失败返回用户 ID 为 00 00 和权限值

注: 程序可以简单判断 2bytesID 是否为 0 来判定是否比对成功, 如果比对成功, 则 2bytesID!=0.

3.8 获取未使用的用户 ID (CMD/ACK 都是 8 字节)

Bytes	1	2	3	4	5	6	7	8
CMD	0xF5	0x0D	0	0	0	0	CHK	0xF5
ACK	0xF5	0x0D	UserID (higher 8 bits)	UserID (lower 8 bits)	ACK_SUCCESS ACK_FAIL	0	CHK	0xF5

说明：模组会返回第一个未使用的用户 ID。

示例：

通过串口发送

F5 0D 00 00 00 0D F5

模组收到指令后，

F5 0D **00 02** 00 0F F5

则表示 0002 号用户 ID 可以使用

3.9 设置串口波特率 (CMD/ACK 都是 8 字节)

Bytes	1	2	3	4	5	6	7	8
CMD	0xF5	0x21	0	New baud rate ID	Flag	0	CHK	0xF5
ACK	0xF5	0x21	0	New baud rate ID	ACK_SUCCUSS ACK_FAIL	0	CHK	0xF5

说明：

串口波特率数字从 1 到 5，对应的波特率是 9600bps 19200bps 38400bps 57600bps 115200bps，每次系统复位，波特率将会恢复到默认的值，默认的波特率是 115200。

Flag=0 时，表示临时修改波特率，模块掉电重启即恢复到修改之前的状态。Flag=1 时，表示永久修改波特率，模块会在非易失存储器记录修改信息，模块重启也不会恢复到修改之前的。

示例：

通过串口发送

F5 21 00 01 00 00 20 F5

设置 9600Bps

模组收到指令后，校验将要设置波特率 ID 是否合法，

合法返回成功

F5 21 00 01 **00** 00 20 F5

不合法返回失败

例如：

通过串口发送

F5 21 00 07 00 00 26 F5

设置波特率不合法，返回

F5 21 00 07 **01** 00 27 F5

3.10 读取图像并提取特征值(CMD 是 8 字节/ ACK >8 字节)

CMD data Format

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x23	0	0	0	0	CHK	0xF5

ACK data Format

i)Data head

Byte	1	2	3	4	5	6	7	8
ACK	0xF5	0x23	Hi (Len)	Low (Le n)	ACK_SUCCESS ACK_FAIL ACK_TIMEOUT ACK_BREAK	0	CHK	0xF5

ii) Data package

Byte	1	2	3	4	5...Len+1	Len+2	Len+3
ACK	0xF5	0	0	0	Feature Data	CHK	0xF5

说明:

采集指纹并提取指纹图像的特征值, 返回值总长度为 8206 字节。

示例:

主机端发送

F5 23 00 00 00 00 22 F5

模组收到指令后, 等待用户按压指纹。

若没有输入指纹, 模组超时 (8s) 退出, 模块端返回超时错误:

F5 23 20 03 08 00 08 F5

当在超时时间内正确按压指纹后, 模块端即按上述格式返回指纹特征值数据包。

特别注意, 此命令也支持打断退出, 即在等待用户按压指纹的时候, 主机端可以重发当前指令让模组退出采集状态。模组返回:

F5 23 20 03 18 00 18 F5

3.11 获取图像(CMD 是 8 字节/ ACK >8 字节)

CMD data Format

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x24	0	0	0	0	CHK	0xF5

ACK data Format

i)Data head

Byte	1	2	3	4	5	6	7	8
ACK	0xF5	0x24	Width> >2	Height >>2	ACK_SUCCESS ACK_FAIL	0	CHK	0xF5

ii) Data package

Byte	1	2---Len+1	Len+2	Len+3
ACK	0xF5	Image Data	CHK	0xF5

说明:

通过串口发送

F5 24 00 00 00 00 24 F5

模组收到指令后，采集图像，采集成功后把图像位图数据送出来。

若超时没有采集到指纹，返回超时指令。

F5 24 2C 2C 08 00 2C F5 （测试时使用 176*176 的传感器）

特别注意，此命令也支持打断退出，即在等待用户按压指纹的时候，主机端可以重发当前指令让模组退出采集状态。模组返回：

F5 24 2C 2C 18 00 3C F5

注：测试时使用 176*176 的传感器

3.12 获取模块版本信息 (CMD 是 8 字节/ ACK >8 字节)

CMD data Format

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x26	0	0	0	0	CHK	0xF5

ACK data Format

i)Data head

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x26	Hi (Len)	Low (Len)	ACK_SUCCESS ACK_FAIL	0	CHK	0xF5

ii)Data package

Byte	1	2...Len+1	Len+2	Len+3
ACK	0xF5	Version data	CHK	0xF5

说明：

获取模组版本信息，其中包含了程序版本、传感器类型、注册方式、编译时间等信息。

3.13 设置/获取比对等级 (CMD/ACK 都是 8 字节)

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x28	0	NEW matchi ng level	0	0	CHK	0xF5
ACK	0xF5	0x28	0	Old matchi ng level	ACK_SUCCESS ACK_FAIL	0	CHK	0xF5

说明：

比对等级可以设置为 0-2，默认为等级 2，比对等级越高，限制越严格。模组重启后会恢复到默认值。

比对等级与相应的误识率关系见下表：

安全等级	误识率
0	1/10w
1	1/50w
2	1/100w

示例:

(设置比对等级 1) 通过串口发送

F5 28 00 01 00 00 29 F5

模组收到指令后, 返回

F5 28 00 01 00 00 29 F5 (说明模组的设置前的比对等级是 1)

3.14 设置/获取注册模式(CMD/ACK 都是 8 字节)

CMD data Format

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x2D	0	0:Allowing same 1:Denying same	0:Set new enrollment mode 1:Get current enrollment mode	0	CHK	0xF5

ACK data Format

Byte	1	2	3	4	5	6	7	8
ACK	0xF5	0x2D	0x01	0:Allowing same 1:Denying same	ACK_SUCCESS ACK_FAIL	0	CHK	0xF5

说明:

指纹注册模式分成两种: 允许重复和拒绝重复模式。

模组上电后, 默认为允许重复注册模式。掉电后, 如过已经设置为拒绝重复模式, 会复位成允许重复注册模式。

注: 在拒绝重复模式下, 在注册时发送第一条注册指令后, 如重复注册已存在的用户会返回 ACK_USER_EXIST 0x07。

示例:

设置拒绝注册重复模式

发送: F5 2D 00 01 00 00 2C F5

返回: F5 2D 01 01 00 00 2D F5

3.15 手指检测功能 (CMD/ACK 都是 8 字节)

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x30	0	0	0	0	CHK	0xF5
ACK	0xF5	0x30	0	0	ACK_SUCCESS ACK_FAIL	0	CHK	0xF5

说明:

此指令只能在未进行注册流程时使用, 发送后立即检测模组上是否有手指按压, 如有手指按压则返回 ACK_SUCCESS, 未有手指按压则返回 ACK_FAIL。

3.16 获取指纹模组唯一 ID 号 (CMD/ACK 都是 8 字节)

CMD data Format

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x60	0	0	0	0	CHK	0xF5

ACK data Format

i) Data Head

Byte	1	2	3	4	5	6	7	8
ACK	0xF5	0x60	0	0	0	0	CHK	0xF5

ii) Data Package

Byte	9	10	11		17	18	19
ACK	0xF5	Fingerprint Module Unique ID Number				CHK	0xF5

说明:

获取指纹模组唯一 ID 号。

发送: F5 60 00 00 00 00 60 F5

返回: F5 60 00 00 00 00 60 F5 F5 **38 30 32 30 39 04 2C 15 0E** F5 红色加粗字体为模组唯一 ID 值, 蓝色加粗为校验位。

3.17 配置注册功能 (CMD/ACK 都是 8 字节)

CMD data Format

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0x3F	PID High	PID Low	PV	0	CHK	0xF5

说明:

PID High: 参数索引高字节。

PID Low: 参数索引低字节。

PV: 参数值。

参数索引列表 (PID):

0x0000: 注册模式 1CNR、NCNR 选择。

0x0001: 同源性校验等级。

0x0002: 抬手检测配置。
0x0003: 指纹采集次数 N。
0x0004: 贴膜攻击检测功能配置。
0x0040: 自学习优先标记。

注：模组上电后，默认为 3C3R 注册模式，同源性校验，无抬手检测。掉电后，如过已经配置过其它注册功能，会复位成默认注册配置。

1. 1CNR/NCNR 注册模式选择：

分为 1CNR、NCNR，N 代表采集指纹的次数。

1CNR: F5 3F 00 00 05 00 3A F5

NCNR: F5 3F 00 00 06 00 39 F5

2. 同源性等级设置（此配置指令可单独发送）：

以 3C3R 注册方式为例，在注册的时候判断 3 次采集到的后两次指纹至少有一枚指纹和第一次指纹保持一致，方可注册成功。

0 的时候不做同源性校验即不判断是否为同一手指，5 的时候要求采集到的指纹完全一致。

等级 0: F5 3F 00 01 00 00 3E F5

等级 1: F5 3F 00 01 01 00 3F F5

等级 2: F5 3F 00 01 02 00 3C F5

等级 3: F5 3F 00 01 03 00 3D F5

等级 4: F5 3F 00 01 04 00 3A F5

等级 5: F5 3F 00 01 05 00 3B F5

3. 抬手检测（此配置指令可单独发送）：

使用抬手检测功能: F5 3F 00 02 01 00 3C F5

4. 采集次数设置：

采集 3 次: F5 3F 00 03 03 00 3F F5

采集 4 次: F5 3F 00 03 04 00 38 F5

采集 5 次: F5 3F 00 03 05 00 39 F5

5. 先返回后自学习（此配置指令可单独发送）：

比对前发送该指令: F5 3F 00 40 00 00 7F F5

注：使用该指令需注意，自学习时间需要 150ms-250ms，在收到比对返回指令后需延迟 300ms 才可对模组掉电，提前掉电会导致指纹自学习功能损坏。

6. 关闭贴膜攻击检测功能（此配置指令可单独发送）：

注册前发送该指令: F5 3F 00 04 00 00 3B F5

注：发送该指令后即可关闭贴膜攻击检测功能，建议在 1CNR 模式下使用。

实例 1：设置注册次数 5 次（1C5R/5C5R）

1C5R 注册方式：先发送指令设置为 1CNR 模式 F5 3F 00 00 05 00 3A F5（成功返回 F5 3F 00 00 00 00 3F F5），然后设置采集次数 F5 3F 00 03 05 00 39 F5（成功返回 F5 3F 00 00 00 00 3F F5）。

5C5R 注册方式：先发送指令设置为 NCNR 模式 F5 3F 00 00 06 00 39 F5（成功返回 F5 3F 00 00 00 00 3F F5），然后设置采集次数 F5 3F 00 03 05 00 39 F5（成功返回 F5 3F 00

00 00 00 3F F5)。

注：注册次数设置必须发送 2 条指令，第 1 条为设置 1CNR 或是 NCNR 模式，第 2 条为设置采集次数。该两条指令只能按顺序依次发送才会有效。

实例 2：设置同源性校验等级（等级 0）

发送指令：F5 3F 00 01 00 00 3E F5（成功返回 F5 3F 00 00 00 00 3F F5）

实例 3：设置注册次数为 6 次（NCNR 模式），同源性校验等级为 0，使用抬手检测功能。

发送指令依次为：

F5 3F 00 00 06 00 39 F5

F5 3F 00 01 00 00 3E F5

F5 3F 00 02 01 00 3C F5

F5 3F 00 03 06 00 3A F5

注：依次发送指令，每条指令成功后会返回 F5 3F 00 00 00 00 3F F5，在收到前 1 条指令的成功返回后再发送下条指令。

3.18 光圈控制命令（CMD/ACK 都是 8 字节）

CMD data Format

i)

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0xC3	led_Start	Led_End	Period	0	CHK	0xF5

ii)ACK data

Byte	1	2	3	4	5	6	7	8
CMD	0xF5	0xC3	0	0	0	0	CHK	0xF5

说明：

Led Start：两种颜色灯切换，起始时灯的颜色；

Led End：两种颜色灯切换，结束时灯的颜色；

Period：一种颜色变到另外一种颜色的周期时间，单位是 10ms，即 Period=100 时，单次颜色变化的时间为 1s；Period 的范围为 30-200，如果超过该范围则该指令不执行。

其中 Start 和 End 分别用 uint8 类型变量的低三位来表示。

0x0000 0 0 0 0 最低三位分别代表 Red Green Blue 三种颜色，低电平有效。

R G B

例如：

0x03 = 0x0000 0011 表示红灯亮

0x05 = 0x0000 0101 表示绿灯亮

0x06 = 0x0000 0110 表示蓝灯亮

0x01 = 0x0000 0001 表示红灯和绿灯的混合颜色

0x02 = 0x0000 0010 表示红灯和蓝灯的混合颜色

示例 1：

呼吸灯

红

F5 C3 03 07 96 00 51 F5

绿

F5 C3 05 07 96 00 57 F5

蓝

F5 C3 06 07 96 00 54 F5

红绿

F5 C3 01 07 96 00 53 F5

红蓝

F5 C3 02 07 96 00 50 F5

绿蓝

F5 C3 04 07 96 00 56 F5

示例 2:

渐变灯

红-蓝

F5 C3 03 06 96 00 50 F5

红-绿

F5 C3 03 05 96 00 53 F5

绿-蓝

F5 C3 05 06 96 00 56 F5

红绿-红蓝

F5 C3 01 02 96 00 56 F5

红绿-绿蓝

F5 C3 01 04 96 00 50 F5

红蓝-绿蓝

F5 C3 02 04 96 00 53 F5

示例 3:

单色灯

红

F5 C3 03 03 96 00 55 F5

绿

F5 C3 05 05 96 00 55 F5

蓝

F5 C3 06 06 96 00 55 F5

红绿

F5 C3 01 01 96 00 55 F5

红蓝

F5 C3 02 02 96 00 55 F5

绿蓝

F5 C3 04 04 96 00 55 F5

关灯（全灭）

F5 C3 07 07 96 00 55 F5

版本历史记录

版本号	修改日期	修改内容	修改人
V1.0	2015.12.31	编制	王磊
V1.1	2016.01.04	修订	赵旭
V1.2	2016.02.25	增加安全等级相关描述	付誉阳
V1.3	2016.03.23	增加永久修改波特率描述	赵旭
V1.4	2016.03.24	修改 1 比 N 比对返回值描述	付誉阳
V1.5	2016.11.24	修改了部分指令的返回值描述	付誉阳
V1.6	2018.09.18	基于 8K-47.8 算法程序下的协议修订	谢鑫阳